



Training Algorithms

Thanh-Sach LE

✉ Itsach@hcmut.edu.vn

Faculty of Computer Science & Engineering
Ho Chi Minh City University of Technology (HCMUT), VNU-HCM

Tp.HCM on August 23, 2025

Training Algorithms

Content

Problem Setup

Training Process

Optimization:
Common Variants

Training
Techniques

Practical
Considerations

Summary

Content

- ① Problem Setup
- ② Training Process
- ③ Optimization: Common Variants
- ④ Training Techniques
- ⑤ Practical Considerations
- ⑥ Summary



Training Algorithms

1 Content

Problem Setup

Training Process

Optimization:
Common Variants

Training
Techniques

Practical
Considerations

Summary



Problem Setup

Training Algorithms

Content

2 **Problem Setup**

Training Process

Optimization:
Common Variants

Training
Techniques

Practical
Considerations

Summary

Training Setup



Training Algorithms

Content

3 Problem Setup

Training Process

Optimization:
Common Variants

Training
Techniques

Practical
Considerations

Summary

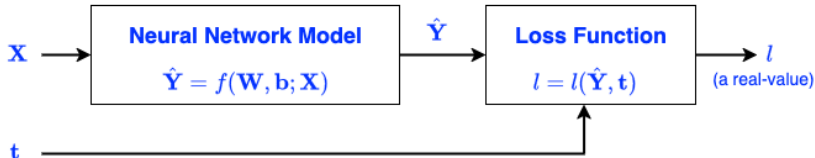
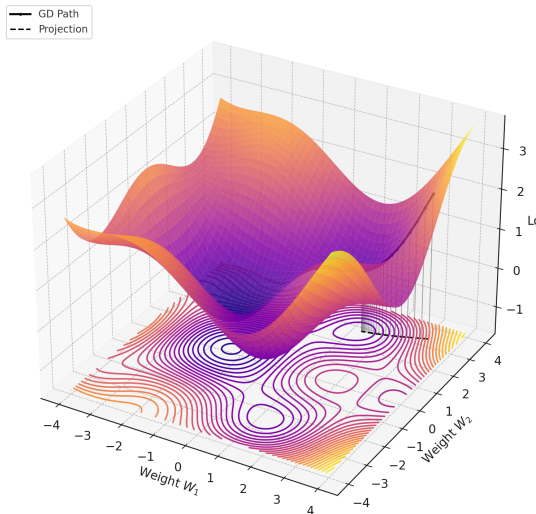


Figure 1.1: Training diagram

- Goal: learn parameters $\theta = \{W, b\}$ to minimize a loss function.
- Typical losses:
 - Regression: Mean Squared Error (MSE).
 - Classification: Binary Cross-Entropy (BCE) or Categorical Cross-Entropy.

Loss Surface

Loss Surface with Gradient Descent Path & Floor Contours



Regression Losses



Training Algorithms

Content

5 Problem Setup

Training Process

Optimization:
Common Variants

Training
Techniques

Practical
Considerations

Summary

Notation: dataset $\{(\mathbf{x}_i, y_i)\}_{i=1}^n$, prediction $\hat{y}_i = f_{\theta}(\mathbf{x}_i)$.

Mean Squared Error (MSE)

$$\mathcal{L}_{\text{MSE}} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Meaning: penalizes squared error, amplifies large mistakes.

Note: sensitive to outliers; suitable if noise $\sim \mathcal{N}(0, \sigma^2)$.

Regression Losses



Training Algorithms

Content

6 Problem Setup

Training Process

Optimization:
Common Variants

Training
Techniques

Practical
Considerations

Summary

Notation: dataset $\{(\mathbf{x}_i, y_i)\}_{i=1}^n$, prediction $\hat{y}_i = f_{\theta}(\mathbf{x}_i)$.

Mean Absolute Error (MAE)

$$\mathcal{L}_{\text{MAE}} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

Meaning: measures absolute error, more robust to outliers.

Note: not differentiable at 0; may converge slower than MSE.

Regression Losses



Training Algorithms

Content

7 Problem Setup

Training Process

Optimization:
Common Variants

Training
Techniques

Practical
Considerations

Summary

Notation: dataset $\{(\mathbf{x}_i, y_i)\}_{i=1}^n$, prediction $\hat{y}_i = f_{\theta}(\mathbf{x}_i)$.

Huber Loss

$$\mathcal{L}_{\delta}(e_i) = \begin{cases} \frac{1}{2}e_i^2 & \text{if } |e_i| \leq \delta \\ \delta(|e_i| - \frac{1}{2}\delta) & \text{if } |e_i| > \delta \end{cases} \quad \text{with } e_i = y_i - \hat{y}_i$$

Meaning: combines MSE (small errors) and MAE (large errors).

Note: hyperparameter δ controls sensitivity to outliers.

Binary Classification Loss



Notation: label $y_i \in \{0, 1\}$, logit $z_i \in \mathbb{R}$, probability $p_i = \sigma(z_i) = \frac{1}{1+e^{-z_i}}$.

Binary Cross-Entropy (BCE)

$$\mathcal{L}_{\text{BCE}} = -\frac{1}{n} \sum_{i=1}^n \left[y_i \log p_i + (1 - y_i) \log(1 - p_i) \right]$$

Meaning: maximizes Bernoulli likelihood; penalizes confident but wrong predictions.

Note: use stable *logits version* to avoid overflow:

$$\mathcal{L}_{\text{BCE-Logits}} = \frac{1}{n} \sum_{i=1}^n \left(\max(0, z_i) - y_i z_i + \log(1 + e^{-|z_i|}) \right)$$

Training Algorithms

Content

8 Problem Setup

Training Process

Optimization:
Common Variants

Training
Techniques

Practical
Considerations

Summary

Binary Classification Loss



Training Algorithms

Content

9 Problem Setup

Training Process

Optimization:
Common Variants

Training
Techniques

Practical
Considerations

Summary

Notation: label $y_i \in \{0, 1\}$, logit $z_i \in \mathbb{R}$, probability $p_i = \sigma(z_i) = \frac{1}{1+e^{-z_i}}$.

- Class imbalance: apply class weights or `pos_weight`.
- Threshold: default 0.5, may tune via ROC/PR.
- Regularization (L2, dropout) helps prevent overfitting.

Multiclass Classification Loss



Training Algorithms

Content

10 Problem Setup

Training Process

Optimization:
Common Variants

Training
Techniques

Practical
Considerations

Summary

Notation: K classes, one-hot label $\mathbf{y}_i \in \{0, 1\}^K$, logits $\mathbf{z}_i \in \mathbb{R}^K$, probabilities $p_{ik} = \text{softmax}(\mathbf{z}_i)_k$.

Categorical Cross-Entropy (CE)

$$\mathcal{L}_{\text{CE}} = -\frac{1}{n} \sum_{i=1}^n \sum_{k=1}^K y_{ik} \log p_{ik} = -\frac{1}{n} \sum_{i=1}^n \log p_{i, y_i}$$

Meaning: maximizes Multinoulli likelihood.

Note: always use *logits version* (softmax+CE combined) for stability.

Multiclass Classification Loss

Notation: K classes, one-hot label $\mathbf{y}_i \in \{0, 1\}^K$, logits $\mathbf{z}_i \in \mathbb{R}^K$, probabilities $p_{ik} = \text{softmax}(\mathbf{z}_i)_k$.

Label Smoothing

Replace one-hot: $y_{i,y_i} = 1 - \varepsilon$, others $y_{ik} = \frac{\varepsilon}{K-1}$.

Meaning: prevents overconfidence, improves generalization.

Focal Loss

$$\mathcal{L}_{\text{Focal}} = -\frac{1}{n} \sum_{i=1}^n \alpha_{y_i} (1 - p_{i,y_i})^\gamma \log p_{i,y_i}$$

Meaning: focuses on hard examples (p_{i,y_i} small).

Note: parameters $\gamma \geq 0$ (focusing) and α_k (class balance).

Training Algorithms

Content

11 Problem Setup

Training Process

Optimization:
Common Variants

Training
Techniques

Practical
Considerations

Summary



Training Process

Training Algorithms

Content

Problem Setup

12 Training Process

Optimization:
Common Variants

Training
Techniques

Practical
Considerations

Summary

Training Process: Overview

- **Goal:** optimize parameters $\theta = \{\mathbf{W}, \mathbf{b}\}$ to minimize loss $\mathcal{L}$.
- Direct minimization is infeasible due to:
 - Non-linear models (e.g., deep networks).
 - High-dimensional parameter space.
- Solution: iterative procedure based on gradients.



Training Algorithms

Content

Problem Setup

13 Training Process

Optimization:
Common Variants

Training
Techniques

Practical
Considerations

Summary

Why Three Steps?

- **Forward pass:** compute predictions $\hat{y}$ and evaluate loss $\mathcal{L}$.
- **Backward pass:** compute gradients $\nabla_{\theta}\mathcal{L}$ via backpropagation.
- **Update step:** adjust parameters θ using an optimizer (e.g., SGD, Adam).

Key Idea

Splitting into 3 steps ensures:

- Clear separation of computation and optimization.
- Efficient reuse of intermediate activations.
- Scalable training for large models.



Training Algorithms

Content

Problem Setup

14 Training Process

Optimization:
Common Variants

Training
Techniques

Practical
Considerations

Summary

Forward Pass

- Input vector: $\mathbf{x}_i$
- Prediction: $\hat{y}_i = f_{\theta}(\mathbf{x}_i)$
- Loss function: $\mathcal{L} = \frac{1}{n} \sum_{i=1}^n \ell(y_i, \hat{y}_i)$

Purpose

- Compute outputs from current parameters.
- Measure discrepancy between prediction and ground truth.



Training Algorithms

Content

Problem Setup

15 Training Process

Optimization:
Common Variants

Training
Techniques

Practical
Considerations

Summary

Backward Pass

- Goal: compute gradients $\nabla_{\theta}\mathcal{L}$.
- Use chain rule through computational graph (backpropagation).

Example

For parameter $w \in \mathbf{W}$:

$$\frac{\partial \mathcal{L}}{\partial w} = \frac{\partial \mathcal{L}}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z} \cdot \frac{\partial z}{\partial w}$$

- Requires saving intermediate activations during forward pass.
- Gradients indicate “direction of steepest ascent” of loss.



Training Algorithms

Content

Problem Setup

16 Training Process

Optimization:
Common Variants

Training
Techniques

Practical
Considerations

Summary

Update Step

- Parameters are updated using optimizer.
- Stochastic Gradient Descent (SGD):**

$$\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}$$

- Adam (adaptive method):**

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) \nabla_{\theta} \mathcal{L}, \quad v_t = \beta_2 v_{t-1} + (1 - \beta_2) (\nabla_{\theta} \mathcal{L})^2$$

$$\theta \leftarrow \theta - \eta \frac{m_t}{\sqrt{v_t + \epsilon}}$$

Purpose

Iteratively refine θ until convergence (loss minimized).



Training Algorithms

Content

Problem Setup

17 Training Process

Optimization:
Common Variants

Training
Techniques

Practical
Considerations

Summary

Pseudocode: SGD Training Algorithm



Inputs

Dataset $\{(\mathbf{x}_i, y_i)\}_{i=1}^n$, model f_{θ} , loss $\ell(\cdot, \cdot)$, learning rate η , epochs E , mini-batch size B .

- 1 Initialize parameters θ .
- 2 **for** epoch = 1 to E **do**
 - 1 Shuffle the dataset and split into mini-batches of size B .
 - 2 **for** each mini-batch $(\mathbf{X}, \mathbf{y})$ **do**
 - 1 *Forward*: $\hat{\mathbf{y}} \leftarrow f_{\theta}(\mathbf{X})$
 - 2 *Loss*: $\mathcal{L} \leftarrow \ell(\mathbf{y}, \hat{\mathbf{y}})$
 - 3 *Backward*: compute gradients $\nabla_{\theta} \mathcal{L}$
 - 4 *Update (SGD)*: $\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}$
 - 3 **end for**
- 3 **end for**

Training Algorithms

Content

Problem Setup

18 Training Process

Optimization:
Common Variants

Training
Techniques

Practical
Considerations

Summary

Notes on Training Terminology



Key Concepts

- **Mini-batch:** a small subset of the training dataset used in one forward-backward-update cycle. Balances efficiency and gradient stability.
- **Epoch:** one complete pass through the entire training dataset (all mini-batches).
- **Iteration (step):** one update of parameters using a single mini-batch.
- **Learning rate η :** step size for parameter updates; too large $\Rightarrow$ divergence, too small $\Rightarrow$ slow convergence.
- **Momentum:** technique to smooth updates by accumulating past gradients, reducing oscillations.
- **Learning rate schedule:** strategy to adjust η during training (e.g., decay, cosine annealing).

Training Algorithms

Content

Problem Setup

19 Training Process

Optimization:
Common Variants

Training
Techniques

Practical
Considerations

Summary



Optimization: Common Variants

Training Algorithms

Content

Problem Setup

Training Process

20

**Optimization:
Common Variants**

Training
Techniques

Practical
Considerations

Summary

63

Optimization Landscape



Method	Update (sketch)	Hyperparams	Pros	Cons / Typical Use
SGD + Momentum	$\mathbf{v}_t = \mu \mathbf{v}_{t-1} + \nabla_{\theta} \mathcal{L}, \theta \leftarrow \theta - \eta \mathbf{v}_t$	η, μ	Simple, strong baselines; good generalization	Sensitive to LR; may oscillate; vision/classic DL
Nesterov (NAG)	$\nabla_{\theta - \eta \mu \mathbf{v}_{t-1}} \mathcal{L}$ (lookahead gradient)	η, μ	Faster than momentum in convex; anticipates step	Extra grad eval; tuning needed
AdaGrad	$\mathbf{G}_t = \mathbf{G}_{t-1} + \mathbf{g}_t^2$; $\theta \leftarrow \theta - \eta \mathbf{g}_t / (\sqrt{\mathbf{G}_t} + \epsilon)$	η, ϵ	Per-parameter adaptivity; sparse data friendly	LR decays too much; NLP (sparse)
RMSProp	$\mathbf{s}_t = \rho \mathbf{s}_{t-1} + (1 - \rho) \mathbf{g}_t^2$; $\theta \leftarrow \theta - \eta \mathbf{g}_t / (\sqrt{\mathbf{s}_t} + \epsilon)$	η, ρ, ϵ	Fixes AdaGrad decay; stable training	Needs schedule; widely used in RNNs
Adam	$\mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1 - \beta_1) \mathbf{g}_t$; $\mathbf{v}_t = \beta_2 \mathbf{v}_{t-1} + (1 - \beta_2) \mathbf{g}_t^2$; $\hat{\mathbf{m}}_t = \mathbf{m}_t / (1 - \beta_1^t)$; $\hat{\mathbf{v}}_t = \mathbf{v}_t / (1 - \beta_2^t)$; $\theta \leftarrow \theta - \eta \hat{\mathbf{m}}_t / (\sqrt{\hat{\mathbf{v}}_t} + \epsilon)$	$\eta, \beta_1, \beta_2, \epsilon$	Fast convergence; default in many tasks	Can overfit; needs weight decay; careful LR
AdamW	Adam + decoupled weight decay: $\theta \leftarrow (1 - \eta \lambda) \theta - \eta \hat{\mathbf{m}}_t / (\sqrt{\hat{\mathbf{v}}_t} + \epsilon)$	$\eta, \beta_1, \beta_2, \epsilon, \lambda$	Better generalization than Adam; modern default	Adds decay hyperparam; transformers/vision
Nadam	Adam + Nesterov momentum (on $\mathbf{m}_t$)	Adam params	Often slightly faster than Adam	Small practical gains; more complex
Adafactor	Factorized 2nd-moment for matrices (memory saving)	η (often schedule)	Low memory; large models (T5)	Fewer guarantees; sensitive schedules
LAMB	AdamW + trust ratio (layer-wise scaling)	AdamW + trust ratio	Large-batch training (1K–64K)	Heavier tuning; large-scale pre-training
SAM	Minimize sharpness: update toward flat minima	Base opt + ρ	Better generalization; robust minima	Extra forward/backward per step; vision/NLP
Lion	Sign-based momentum; update uses $\text{sign}(\mathbf{m}_t)$	η, β_1, β_2	Memory-light; strong on vision	Newer; tuning may vary by task

Training Algorithms

Content

Problem Setup

Training Process

21 Optimization: Common Variants

Training Techniques

Practical Considerations

Summary

Notes

Popularity: Adam/AdamW $\rightarrow$ default in modern practice; SGD+Momentum still strong in vision.
Factors: batch size, weight decay, schedule (cosine, warmup), memory budget, data sparsity.

SGD with Momentum — Update



Formula

$$\mathbf{g}_t = \nabla_{\theta} \mathcal{L}_t(\theta), \quad \mathbf{v}_t = \mu \mathbf{v}_{t-1} + \mathbf{g}_t, \quad \theta \leftarrow \theta - \eta \mathbf{v}_t$$

- θ : parameter vector (e.g., $\{\mathbf{W}, \mathbf{b}\}$).
- $\mathbf{g}_t$: gradient on current mini-batch.
- $\mathbf{v}_t$: momentum (velocity) vector.
- **Initialization:** set $\mathbf{v}_0 = \mathbf{0}$ at the beginning of training.

Note

$\mathbf{v}_t$ is recursive: each update depends on previous momentum $\mathbf{v}_{t-1}$. Initialization with zero is standard; after several steps, $\mathbf{v}_t$ accumulates a smoothed history of past gradients.

Training Algorithms

Content

Problem Setup

Training Process

22

Optimization:
Common Variants

Training
Techniques

Practical
Considerations

Summary

SGD with Momentum — Hyperparameters



Training Algorithms

Content

Problem Setup

Training Process

23

Optimization:
Common Variants

Training
Techniques

Practical
Considerations

Summary

Terms

- μ : momentum coefficient, scalar. Typical range: 0.8–0.99 (commonly 0.9).
- η : learning rate, scalar. Task-dependent; try 10^{-3} – 10^{-1} with schedule (cosine, step).
- λ : weight decay (optional). Often 10^{-4} in CNN training from scratch.

SGD with Momentum — Intuition



Training Algorithms

Content

Problem Setup

Training Process

24

**Optimization:
Common Variants**

Training
Techniques

Practical
Considerations

Summary

Why Momentum?

- Accumulates a running direction (inertia).
- Dampens oscillations across steep directions.
- Accelerates along shallow valleys.
- More stable than vanilla SGD in ill-conditioned landscapes.

SGD with Momentum — Usage



Training Algorithms

Content

Problem Setup

Training Process

25

Optimization:
Common Variants

Training
Techniques

Practical
Considerations

Summary

Practical Notes

- Strong baseline for CNNs; often generalizes better than adaptive methods.
- Pair with cosine decay + warmup for learning rate.
- Tune η first, then adjust μ and λ .
- Scale η with batch size (linear rule as heuristic).

Nesterov Accelerated Gradient (NAG) — Update



Training Algorithms

Content

Problem Setup

Training Process

26

Optimization:
Common Variants

Training
Techniques

Practical
Considerations

Summary

Formula

$$\mathbf{g}_t = \nabla_{\theta} \mathcal{L}(\theta - \eta \mu \mathbf{v}_{t-1}), \quad \mathbf{v}_t = \mu \mathbf{v}_{t-1} + \mathbf{g}_t, \quad \theta \leftarrow \theta - \eta \mathbf{v}_t$$

- θ : parameter vector.
- $\mathbf{g}_t$: gradient evaluated at the *lookahead* position.
- $\mathbf{v}_t$: momentum (velocity) vector.
- **Initialization:** set $\mathbf{v}_0 = \mathbf{0}$.

Nesterov Accelerated Gradient (NAG) — Hyperparameters



Training Algorithms

Content

Problem Setup

Training Process

27

Optimization:
Common Variants

Training
Techniques

Practical
Considerations

Summary

Terms

- μ : momentum coefficient, scalar. Typical range: 0.8–0.99 (commonly 0.9).
- η : learning rate, scalar. Similar scale as in SGD with momentum; tune carefully.

Nesterov Accelerated Gradient (NAG) — Intuition



Training Algorithms

Content

Problem Setup

Training Process

28

Optimization:
Common Variants

Training
Techniques

Practical
Considerations

Summary

Why Nesterov?

- Anticipates the effect of the momentum step (lookahead).
- Provides a correction before moving, often leading to faster convergence.
- Reduces overshoot compared to classical momentum.

Nesterov Accelerated Gradient (NAG) — Usage



Training Algorithms

Content

Problem Setup

Training Process

29

**Optimization:
Common Variants**

Training
Techniques

Practical
Considerations

Summary

Practical Notes

- Often faster than vanilla momentum in convex and smooth settings.
- More sensitive to learning rate η ; careful tuning needed.
- Used in vision tasks and sometimes in RNN training.
- Common default: $\mu = 0.9$, cosine LR decay with warmup.

AdaGrad — Update



Training Algorithms

Content

Problem Setup

Training Process

30

Optimization:
Common Variants

Training
Techniques

Practical
Considerations

Summary

Formula

$$\mathbf{G}_t = \mathbf{G}_{t-1} + \mathbf{g}_t^2, \quad \theta \leftarrow \theta - \eta \frac{\mathbf{g}_t}{\sqrt{\mathbf{G}_t} + \epsilon}$$

- θ : parameter vector.
- $\mathbf{g}_t$: gradient at step t (element-wise squared in $\mathbf{g}_t^2$).
- $\mathbf{G}_t$: accumulated sum of squared gradients (per parameter).
- **Initialization:** set $\mathbf{G}_0 = \mathbf{0}$.

AdaGrad — Hyperparameters



Training Algorithms

Content

Problem Setup

Training Process

31 Optimization:
Common Variants

Training
Techniques

Practical
Considerations

Summary

Terms

- η : global learning rate (scalar). Typical start: 0.01 or 0.1.
- ϵ : small constant for numerical stability, e.g. 10^{-8} .

AdaGrad — Intuition



Training Algorithms

Content

Problem Setup

Training Process

32

Optimization:
Common Variants

Training
Techniques

Practical
Considerations

Summary

Why AdaGrad?

- Adapts learning rate for each parameter individually.
- Frequent features $\Rightarrow$ smaller effective step size. Infrequent features $\Rightarrow$ larger effective step size.
- Naturally well-suited for sparse data (e.g., text / NLP).

AdaGrad — Usage



Training Algorithms

Content

Problem Setup

Training Process

33

Optimization:
Common Variants

Training
Techniques

Practical
Considerations

Summary

Practical Notes

- Good choice for sparse features or embeddings.
- Main drawback: effective learning rate decays monotonically, can stall long training.
- Rarely used for deep networks today; mostly replaced by RMSProp/Adam.
- Still relevant in NLP and recommendation tasks with sparse gradients.

RMSProp — Update



Training Algorithms

Content

Problem Setup

Training Process

34

Optimization:
Common Variants

Training
Techniques

Practical
Considerations

Summary

Formula

$$\mathbf{s}_t = \rho \mathbf{s}_{t-1} + (1 - \rho) \mathbf{g}_t^2, \quad \boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \frac{\mathbf{g}_t}{\sqrt{\mathbf{s}_t} + \epsilon}$$

- $\boldsymbol{\theta}$: parameter vector.
- $\mathbf{g}_t$: gradient at step t .
- $\mathbf{s}_t$: exponentially decayed average of squared gradients.
- **Initialization:** set $\mathbf{s}_0 = \mathbf{0}$.

RMSProp — Hyperparameters



Training Algorithms

Content

Problem Setup

Training Process

35

Optimization:
Common Variants

Training
Techniques

Practical
Considerations

Summary

Terms

- η : learning rate (scalar). Typical range: $10^{-4} - 10^{-2}$.
- ρ : decay rate (forgetting factor). Commonly 0.9.
- ϵ : small constant for stability, e.g. 10^{-8} .

RMSProp — Intuition



Training Algorithms

Content

Problem Setup

Training Process

36

**Optimization:
Common Variants**

Training
Techniques

Practical
Considerations

Summary

Why RMSProp?

- Fixes AdaGrad's issue of monotonically decaying learning rates.
- Maintains a moving average of squared gradients.
- Effectively adapts learning rate per parameter in non-stationary problems.
- Provides more stable training for deep neural nets.

RMSPprop — Usage



Training Algorithms

Content

Problem Setup

Training Process

37

**Optimization:
Common Variants**

Training
Techniques

Practical
Considerations

Summary

Practical Notes

- Well-suited for non-stationary objectives (e.g., RNN training).
- Popular choice before Adam; still used in reinforcement learning.
- Tune η and ρ carefully; defaults often $\eta = 0.001$, $\rho = 0.9$.
- Consider combining with momentum for improved stability.

Adam — Update



Formula

$$\mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1 - \beta_1) \mathbf{g}_t, \quad \mathbf{v}_t = \beta_2 \mathbf{v}_{t-1} + (1 - \beta_2) \mathbf{g}_t^2$$

$$\hat{\mathbf{m}}_t = \frac{\mathbf{m}_t}{1 - \beta_1^t}, \quad \hat{\mathbf{v}}_t = \frac{\mathbf{v}_t}{1 - \beta_2^t}, \quad \theta \leftarrow \theta - \eta \frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{v}}_t} + \epsilon}$$

- θ : parameter vector.
- $\mathbf{g}_t$: gradient at step t .
- $\mathbf{m}_t$: exponential moving average of gradients (first moment).
- $\mathbf{v}_t$: exponential moving average of squared gradients (second moment).
- $\hat{\mathbf{m}}_t, \hat{\mathbf{v}}_t$: bias-corrected estimates.
- **Initialization:** $\mathbf{m}_0 = \mathbf{0}, \mathbf{v}_0 = \mathbf{0}$.

Training Algorithms

Content

Problem Setup

Training Process

38

Optimization:
Common Variants

Training
Techniques

Practical
Considerations

Summary

63

Adam — Hyperparameters



Training Algorithms

Content

Problem Setup

Training Process

39

**Optimization:
Common Variants**

Training
Techniques

Practical
Considerations

Summary

Typical Settings

- η : learning rate (default 0.001).
- β_1 : decay for first moment (default 0.9).
- β_2 : decay for second moment (default 0.999).
- ϵ : stability constant (default 10^{-8}).

Adam — Intuition



Training Algorithms

Content

Problem Setup

Training Process

40

**Optimization:
Common Variants**

Training
Techniques

Practical
Considerations

Summary

Why Adam?

- Combines benefits of Momentum (first moment) and RMSProp (second moment).
- Adaptive per-parameter learning rates.
- Bias-correction ensures stability in early iterations.
- Usually fast and robust across many tasks.

Adam — Usage



Practical Notes

- **Default optimizer in many frameworks (PyTorch, TensorFlow).**
- Works well out-of-the-box for NLP and RL tasks.
- Can overfit or lead to poor generalization without weight decay.
- Often combined with learning rate schedules (cosine decay, warmup).
- For vision models (CNNs, transformers), AdamW (decoupled weight decay) is preferred.

Training Algorithms

Content

Problem Setup

Training Process

41

**Optimization:
Common Variants**

Training
Techniques

Practical
Considerations

Summary

AdamW — Update



Formula

$$\theta \leftarrow (1 - \eta\lambda) \theta - \eta \frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{v}}_t + \epsilon}}$$

- $\hat{\mathbf{m}}_t$: bias-corrected first moment estimate.
- $\hat{\mathbf{v}}_t$: bias-corrected second moment estimate.
- η : learning rate.
- λ : weight decay coefficient.
- **Initialization:** same as Adam ($\mathbf{m}_0 = \mathbf{0}, \mathbf{v}_0 = \mathbf{0}$).

Training Algorithms

Content

Problem Setup

Training Process

42

Optimization:
Common Variants

Training
Techniques

Practical
Considerations

Summary

63

AdamW — Hyperparameters



Training Algorithms

Content

Problem Setup

Training Process

43

**Optimization:
Common Variants**

Training
Techniques

Practical
Considerations

Summary

Typical Settings

- η : default 10^{-3} (transformers often 2×10^{-4} to 5×10^{-5}).
- $\beta_1 = 0.9$, $\beta_2 = 0.999$ (same as Adam).
- $\epsilon = 10^{-8}$ for stability.
- λ : weight decay, often 0.01 for large models.

AdamW — Intuition



Training Algorithms

Content

Problem Setup

Training Process

44

Optimization:
Common Variants

Training
Techniques

Practical
Considerations

Summary

Why AdamW?

- In Adam, adding L2 regularization inside the loss couples it with adaptive step sizes $\rightarrow$ suboptimal.
- AdamW decouples weight decay from the gradient update.
- This yields better control of regularization and significantly improves generalization.

AdamW — Usage



Practical Notes

- **Default optimizer for transformers (e.g., BERT, GPT) and modern vision backbones.**
- Usually paired with cosine learning rate schedule + warmup steps.
- Tune weight decay carefully (typical 0.01, but can vary).
- Outperforms Adam in most large-scale training scenarios.

Training Algorithms

Content

Problem Setup

Training Process

45

**Optimization:
Common Variants**

Training
Techniques

Practical
Considerations

Summary

63

Nadam

Idea: Nesterov-style correction on Adam's momentum.

Effect: sometimes slightly faster than Adam in practice.

Tip: start from Adam's best LR/schedule; small gains, task-dependent.



Training Algorithms

Content

Problem Setup

Training Process

46

Optimization:
Common Variants

Training
Techniques

Practical
Considerations

Summary

63

Adafactor (Memory-Efficient)

Factorized second moment for matrices (approx. $\mathbf{v}_t$ by row/col stats).

Pros: drastically reduced memory; suitable for very large models.

Notes: pair with LR schedule (no fixed η); used in T5 training.



Training Algorithms

Content

Problem Setup

Training Process

47

Optimization:
Common Variants

Training
Techniques

Practical
Considerations

Summary

LAMB (Large-Batch Training)

Trust ratio: layer-wise scaling for stable updates at huge batch sizes.

Use: pretraining with batch $> 1k$; combine with warmup + cosine decay.

Caveat: more hyperparams; verify accuracy parity with smaller-batch baselines.



Training Algorithms

Content

Problem Setup

Training Process

48

Optimization:
Common Variants

Training
Techniques

Practical
Considerations

Summary

SAM: Sharpness-Aware Minimization

Objective: prefer flat minima by penalizing sharpness.

Mechanism: inner ascent to worst-case in a neighborhood, then descent.

Pros: better generalization; **Cost:** $\approx 2\times$ compute per step.

Variants: ASAM, GSAM; can wrap around SGD/AdamW.



Training Algorithms

Content

Problem Setup

Training Process

49

Optimization:
Common Variants

Training
Techniques

Practical
Considerations

Summary

63

Lion (Sign-Based Momentum)

Update: uses $\text{sign}(\mathbf{m}_t)$ for parameter step (memory-light).

Pros: strong on vision; reduces second-moment storage.

Notes: newer; compare to AdamW on your task.



Training Algorithms

Content

Problem Setup

Training Process

50

Optimization:
Common Variants

Training
Techniques

Practical
Considerations

Summary

63

Summary & Choosing the Optimizer



Training Algorithms

Content

Problem Setup

Training Process

51

Optimization:
Common Variants

Training
Techniques

Practical
Considerations

Summary

- **General default:** AdamW + cosine LR + warmup; tune weight decay.
- **Vision (from-scratch CNNs):** SGD + Momentum often best generalization.
- **Large-batch pretraining:** LAMB (or AdamW + careful scaling) + warmup.
- **Sparse features / NLP classic:** AdaGrad / RMSProp still relevant.
- **Flat minima / robustness:** SAM (wrap your base optimizer).
- **Memory-limited large models:** Adafactor; **New trials:** Lion.

Practical Tips

Schedule matters (cosine, step, one-cycle); always try warmup.

Tune η first, then decay/momentum; log training curves and validate frequently.



Training Techniques

Training Algorithms

Content

Problem Setup

Training Process

Optimization:
Common Variants

52

Training
Techniques

Practical
Considerations

Summary

63

Training Techniques



Training Algorithms

Content

Problem Setup

Training Process

Optimization:
Common Variants

53 Training
Techniques

Practical
Considerations

Summary

- **Learning rate scheduling:** step decay, cosine annealing, warm restarts.
- **Regularization:**
 - L2 (weight decay).
 - Dropout.
- **Normalization:** BatchNorm, LayerNorm.
- **Early stopping:** stop when validation error increases.

Learning Rate Scheduling



Training Algorithms

Content

Problem Setup

Training Process

Optimization:
Common Variants

54 Training
Techniques

Practical
Considerations

Summary

Idea

Adjust η dynamically during training to balance exploration vs. convergence.

- **Step decay:** reduce η by a factor every k epochs. Common in CNN training.
- **Cosine annealing:** $\eta_t = \frac{1}{2}\eta_0(1 + \cos(\frac{t}{T}\pi))$. Smooth decay; often with warmup.
- **Warm restarts:** periodically reset η to a higher value $\rightarrow$ escape sharp minima.

Notes

Schedules strongly affect convergence speed and generalization.

Regularization



Why?

Prevent overfitting by constraining model capacity.

- **L2 penalty (weight decay):** Add $\frac{\lambda}{2} \|\theta\|^2$ to loss. Equivalent to multiplicative shrinkage in AdamW. Typical λ : 10^{-4} – 10^{-2} .
- **Dropout:** Randomly set activations to zero during training. Forces network to not rely on single neurons. Drop prob. $p = 0.1$ – 0.5 .

Notes

Both methods can be combined; dropout less used in modern transformers (replaced by normalization + data augmentation).

Training Algorithms

Content

Problem Setup

Training Process

Optimization:
Common Variants

55 Training
Techniques

Practical
Considerations

Summary

Normalization



Why?

Stabilize training by reducing internal covariate shift.

- **BatchNorm**: normalize activations across batch. Works well for CNNs; depends on batch size.
- **LayerNorm**: normalize across features per sample. Common in transformers; independent of batch size.

Notes

Normalization smooths the optimization landscape, allows larger η , and improves convergence.

Training Algorithms

Content

Problem Setup

Training Process

Optimization:
Common Variants

56

Training
Techniques

Practical
Considerations

Summary

63

Early Stopping



Training Algorithms

Content

Problem Setup

Training Process

Optimization:
Common Variants

57

Training
Techniques

Practical
Considerations

Summary

Idea

Stop training when validation error no longer decreases (to avoid overfitting).

- Monitor validation loss or accuracy each epoch.
- If no improvement for p epochs (patience), stop training.
- Save best model weights (checkpointing).

Notes

Simple and effective; acts as a form of implicit regularization.



Practical Considerations

Training Algorithms

Content

Problem Setup

Training Process

Optimization:
Common Variants

Training
Techniques

58

**Practical
Considerations**

Summary

63

Practical Considerations — Initialization



Initialization

- Proper weight initialization stabilizes variance across layers.
- **Xavier (Glorot)**: suitable for sigmoid/tanh activations.
- **He (Kaiming)**: designed for ReLU and variants.

Training Algorithms

Content

Problem Setup

Training Process

Optimization:
Common Variants

Training
Techniques

59

Practical
Considerations

Summary

63

Practical Considerations — Gradients



Training Algorithms

Content

Problem Setup

Training Process

Optimization:
Common Variants

Training
Techniques

60

Practical
Considerations

Summary

Vanishing Gradients

- Gradients shrink as they backprop through deep networks.
- Mitigation: ReLU activations, residual connections, normalization layers.

Exploding Gradients

- Gradients grow uncontrollably, causing unstable updates.
- Mitigation: gradient clipping, careful initialization.

63

Practical Considerations — Batch Size



Training Algorithms

Content

Problem Setup

Training Process

Optimization:
Common Variants

Training
Techniques

61 Practical
Considerations

Summary

Batch Size Trade-off

- **Small batches:** noisier updates, slower throughput, often better generalization.
- **Large batches:** faster throughput, smoother gradients, but risk poorer generalization. Requires careful learning rate scaling (linear scaling rule).



Summary

Training Algorithms

Content

Problem Setup

Training Process

Optimization:
Common Variants

Training
Techniques

Practical
Considerations

62 Summary

Training Neural Networks: Summary



Key Points

- Training loop consists of:
 - **Forward pass:** compute predictions.
 - **Loss computation:** measure error vs. ground truth.
 - **Backward pass:** compute gradients (backpropagation).
 - **Update step:** adjust weights with optimizer.
- Choice of optimizer, learning rate schedule, and regularization strongly influences convergence and generalization.
- Validation monitoring is essential to detect overfitting and apply early stopping.

Pipeline:

Data → Forward → Loss → Backward → Update → Repeat

Training Algorithms

Content

Problem Setup

Training Process

Optimization:
Common Variants

Training
Techniques

Practical
Considerations

63 Summary